

■ LEVEL 5 • PREFIX SUM

Pattern Recognition • Problem Practice • Optimal C++ • Complexity

Pattern Trigger	Many range sums or subarray conditions. Store cumulative state so earlier work is reused.
Reference Examples	Range Sum Query • Subarray Sum Equals K • Equilibrium/Pivot Index
Problems	6 focused problems in this level
Code Standard	C++ • visible nested indentation • syntax-colored keywords/functions/comments/strings

Reference framework: the user's **DSA Approach Handbook**. The handbook defines the trigger, why, key thought, complexity, and representative practice problems; this continuation turns each level into a problem-by-problem practice book.

LEVEL 5 • EXPANDED PROBLEM INDEX

The original 6 core problems are retained. Four additional problems are added below, giving 10 total problems in this level.

#	Problem	Pattern	LeetCode
1	Range Sum Query - Immutable	Prefix Sum	#303
2	Subarray Sum Equals K	Prefix Sum + HashMap	#560
3	Find Pivot Index	Prefix Sum	#724
4	Continuous Subarray Sum	Prefix Remainder	#523
5	Maximum Subarray	Prefix/DP State	#53
6	Count Number of Nice Subarrays	Prefix Sum + Frequency	#1248
7	Range Sum Query - Immutable	Prefix Sum	303
8	Find Pivot Index	Prefix Balance	724
9	Continuous Subarray Sum	Prefix Remainder	523
10	Product of Array Except Self	Prefix + Suffix	238

Extension rule: Problems 7–10 are additional practice chosen to strengthen the same pattern. They are clearly separated from the reference handbook's original representative set.

PROBLEM 1 Range Sum Query - Immutable

Pattern: Prefix Sum

LeetCode: #303

■ Input	integer array and range queries
■ Output	sum of elements from left to right for each query
■ Constraints	$1 \leq n \leq 10^4$
■ Main Idea / Pattern	Precompute $\text{prefix}[i]$ = sum of elements before i, so a range is $\text{prefix}[r+1] - \text{prefix}[l]$.
■ Time Complexity	$O(n)$ build, $O(1)$ query
■ Space Complexity	$O(n)$

■ Optimal C++ Code

```
class NumArray {
    vector<long long> prefix;
public:
    NumArray(vector<int>& nums) {
        prefix.resize(nums.size() + 1);
        for (int i = 0; i < nums.size(); i++)
            prefix[i + 1] = prefix[i] + nums[i];
    }

    long long sumRange(int left, int right) {
        return prefix[right + 1] - prefix[left];
    }
};
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 2 Subarray Sum Equals K

Pattern: Prefix Sum + HashMap

LeetCode: #560

■ Input	integer array and k
■ Output	number of contiguous subarrays whose sum equals k
■ Constraints	n up to $2 * 10^4$
■ Main Idea / Pattern	If current prefix is P, we need an earlier prefix P-k. Store prefix frequencies.
■■ Time Complexity	O(n)
■ Space Complexity	O(n)

■ Optimal C++ Code

```
int subarraySum(vector<int>& nums, int k) {
    unordered_map<long long, int> freq;
    freq[0] = 1;

    long long prefix = 0;
    int ans = 0;

    for (int x : nums) {
        prefix += x;

        if (freq.count(prefix - k))
            ans += freq[prefix - k];

        freq[prefix]++;
    }

    return ans;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 3 Find Pivot Index

Pattern: Prefix Sum

LeetCode: #724

■ Input	integer array
■ Output	index where left sum equals right sum
■ Constraints	$1 \leq n \leq 10^4$
■ Main Idea / Pattern	Keep total sum and a running left sum. Right sum is total-left-current.
■■ Time Complexity	$O(n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
int pivotIndex(vector<int>& nums) {
    long long total = accumulate(nums.begin(), nums.end(), 0LL);
    long long left = 0;
    for (int i = 0; i < nums.size(); i++) {
        long long right = total - left - nums[i];
        if (left == right)
            return i;
        left += nums[i];
    }
    return -1;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 4 Continuous Subarray Sum

Pattern: Prefix Remainder

LeetCode: #523

■ Input	integer array and k
■ Output	whether a subarray of length at least 2 has sum divisible by k
■ Constraints	k may be zero
■ Main Idea / Pattern	Equal prefix remainders imply their difference is divisible by k. Store the earliest index of each remainder.
■ Time Complexity	O(n)
■ Space Complexity	O(k)

■ Optimal C++ Code

```
bool checkSubarraySum(vector<int>& nums, int k) {
    unordered_map<int, int> first;
    first[0] = -1;

    long long sum = 0;
    for (int i = 0; i < nums.size(); i++) {
        sum += nums[i];

        if (k != 0)
            sum %= k;

        if (first.count(sum)) {
            if (i - first[sum] >= 2) return true;
        } else {
            first[sum] = i;
        }
    }
    return false;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 5 Maximum Subarray

Pattern: Prefix/DP State

LeetCode: #53

■ Input	integer array
■ Output	maximum possible sum of a non-empty contiguous subarray
■ Constraints	$1 \leq n \leq 10^5$
■ Main Idea / Pattern	Track the best subarray ending at the current position. This is Kadane's state transition.
■ Time Complexity	$O(n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
int maxSubArray(vector<int>& nums) {  
    int current = nums[0], best = nums[0];  
    for (int i = 1; i < nums.size(); i++) {  
        current = max(nums[i], current + nums[i]);  
        best = max(best, current);  
    }  
    return best;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 6 Count Number of Nice Subarrays

Pattern: Prefix Sum + Frequency

LeetCode: #1248

■ Input	binary/odd-even integer array and k
■ Output	number of subarrays containing exactly k odd numbers
■ Constraints	$1 \leq n \leq 5 * 10^4$
■ Main Idea / Pattern	Convert each value to 0/1 by parity. Count earlier prefix odd-count values equal to current-k.
■ Time Complexity	O(n)
■ Space Complexity	O(n)

■ Optimal C++ Code

```
int numberOfSubarrays(vector<int>& nums, int k) {  
    unordered_map<int, int> freq;  
    freq[0] = 1;  
  
    int odd = 0, ans = 0;  
    for (int x : nums) {  
        odd += x % 2;  
  
        if (freq.count(odd - k))  
            ans += freq[odd - k];  
  
        freq[odd]++;  
    }  
    return ans;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 7 Range Sum Query - Immutable

Pattern: Prefix Sum

LeetCode: 303

Input	nums, left, right
Output	sum of nums[left..right]
Constraints	1 <= nums.size() <= 1e4
Main Idea / Pattern	Precompute cumulative sums so every range query is answered by subtraction.
Time Complexity	O(n) build, O(1) query
Space Complexity	O(n)

Optimal C++ Code

```
class NumArray {
    vector<int> prefix;
public:
    NumArray(vector<int>& nums) {
        prefix.resize(nums.size() + 1);
        for (int i = 0; i < nums.size(); i++) {
            prefix[i + 1] = prefix[i] + nums[i];
        }
    }
    int sumRange(int left, int right) {
        return prefix[right + 1] - prefix[left];
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 8 Find Pivot Index

Pattern: Prefix Balance

LeetCode: 724

Input	nums
Output	index where left sum equals right sum
Constraints	1 <= nums.size() <= 1e4
Main Idea / Pattern	Keep total sum and a running left sum; right sum is total - left - nums[i].
Time Complexity	O(n)
Space Complexity	O(1)

Optimal C++ Code

```
class Solution {
public:
    int pivotIndex(vector<int>& nums) {
        int total = accumulate(nums.begin(), nums.end(), 0);
        int leftSum = 0;
        for (int i = 0; i < nums.size(); i++) {
            int rightSum = total - leftSum - nums[i];
            if (leftSum == rightSum) return i;
            leftSum += nums[i];
        }
        return -1;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 9 Continuous Subarray Sum

Pattern: Prefix Remainder

LeetCode: 523

Input	nums, k
Output	whether a length-at-least-2 subarray has sum divisible by k
Constraints	1 <= nums.size() <= 1e5
Main Idea / Pattern	Store the first index for each prefix-sum remainder. Equal remainders mean the difference is divisible by k.
Time Complexity	O(n)
Space Complexity	O(k)

Optimal C++ Code

```
class Solution {
public:
    bool checkSubarraySum(vector<int>& nums, int k) {
        unordered_map<int, int> first;
        first[0] = -1;
        int sum = 0;
        for (int i = 0; i < nums.size(); i++) {
            sum = (sum + nums[i]) % k;
            if (first.count(sum)) {
                if (i - first[sum] >= 2) return true;
            } else {
                first[sum] = i;
            }
        }
        return false;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 10 Product of Array Except Self

Pattern: Prefix + Suffix

LeetCode: 238

Input	nums
Output	array where ans[i] is the product of all elements except nums[i]
Constraints	2 <= nums.size() <= 1e5
Main Idea / Pattern	Build prefix products into the answer, then multiply by a running suffix product.
Time Complexity	O(n)
Space Complexity	O(1) extra (excluding output)

Optimal C++ Code

```
class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {
        vector<int> ans(nums.size(), 1);
        int prefix = 1;
        for (int i = 0; i < nums.size(); i++) {
            ans[i] = prefix;
            prefix *= nums[i];
        }
        int suffix = 1;
        for (int i = nums.size() - 1; i >= 0; i--) {
            ans[i] *= suffix;
            suffix *= nums[i];
        }
        return ans;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.